

Data Structures

CSC 301

Lecture Notes

Chapter 1: Data Types and Memory Representation

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

1	Data Types and Memory Representation	2
1.1	Introduction	2
1.2	Primitive Data Types	2
1.2.1	Integer Types	2
1.2.2	Floating-Point Types	2
1.2.3	Character Types	3
1.2.4	Boolean Types	3
1.3	Memory Representation	3
1.3.1	Binary Representation	3
1.3.2	Memory Addressing	3
1.3.3	Endianness	3
1.3.4	Memory Alignment and Padding	4
1.4	Data Representation in Memory	4
1.4.1	Two's Complement Representation	4
1.4.2	IEEE 754 Floating-Point Representation	4
1.4.3	Character Encoding	4
1.5	Conclusion	5

Chapter 1

Data Types and Memory Representation

1.1 Introduction

Data structures are built upon the fundamental data types provided by a programming language and the way these types are represented in computer memory. Understanding primitive data types and their memory representation is essential for designing efficient data structures and algorithms. This chapter introduces the basic building blocks of data — primitive types — and examines how they are stored, addressed, and organized in memory.

1.2 Primitive Data Types

Primitive Data Types are the basic data types directly supported by a programming language. They represent simple values and are not composed of other types. Common categories include integers, floating-point numbers, characters, and booleans.

1.2.1 Integer Types

Integer types represent whole numbers. They vary in size (number of bits) and whether they are signed or unsigned. Typical sizes are 8, 16, 32, and 64 bits. Signed integers use the most significant bit as a sign bit; the most common representation for negative values is **Two's Complement** (see Section 1.4.1).

1.2.2 Floating-Point Types

Floating-point types represent real numbers with fractional parts. They follow the **IEEE 754** standard, which defines single precision (32 bits) and double precision (64 bits) formats. A floating-point number consists of a sign bit, an exponent, and a mantissa (significand).

1.2.3 Character Types

Character types store individual characters. Historically, a single byte (8 bits) was used with the ASCII encoding (0–127). Modern systems use Unicode, which may require multiple bytes per character (e.g., UTF-8, UTF-16).

1.2.4 Boolean Types

Boolean types represent logical values: `true` and `false`. They are typically stored as a single byte or even a single bit when packed into larger structures.

1.3 Memory Representation

Memory Representation refers to how data values are encoded as bits and organized in the addressable memory of a computer. Key concepts include binary encoding, memory addressing, endianness, and alignment.

1.3.1 Binary Representation

All data in a digital computer is ultimately represented as sequences of bits (0 and 1). The interpretation of a bit pattern depends on the data type. For example, the 8-bit pattern 01000001 can represent the integer 65, the character 'A' in ASCII, or a fragment of a larger value.

1.3.2 Memory Addressing

Memory is organized as a sequence of addressable units, typically bytes (8 bits). Each byte has a unique address. Multi-byte data types occupy consecutive addresses. The address of a variable is the address of its first byte.

1.3.3 Endianness

When a multi-byte value is stored in memory, the order of bytes matters. In **big-endian** systems, the most significant byte is stored at the lowest address. In **little-endian** systems, the least significant byte is stored at the lowest address. Most modern processors (x86, ARM) use little-endian, but network protocols often use big-endian (network byte order).

Example: 32-bit integer 0x12345678 stored at address 0x1000

Big-endian:

Address: 0x1000 0x1001 0x1002 0x1003

Byte: 0x12 0x34 0x56 0x78

Little-endian:

Address: 0x1000 0x1001 0x1002 0x1003

Byte: 0x78 0x56 0x34 0x12

1.3.4 Memory Alignment and Padding

Many architectures require that data of size n bytes be stored at addresses that are multiples of n (e.g., a 4-byte integer at an address divisible by 4). This is called **Memory Alignment**. To satisfy alignment, compilers may insert **Padding** bytes between structure fields or at the end of a structure. Padding increases memory usage but can improve access speed and prevent hardware exceptions.

1.4 Data Representation in Memory

This section details the specific bit-level representation of common primitive types.

1.4.1 Two's Complement Representation

Two's Complement is the standard method for representing signed integers. For an n -bit number, the range is $[-2^{n-1}, 2^{n-1} - 1]$. To negate a number, invert all bits and add 1. This representation allows addition and subtraction to be performed identically for signed and unsigned operands.

Example (8-bit):

- +5: 00000101
- -5: invert 00000101 $\rightarrow$ 11111010, add 1 $\rightarrow$ 11111011

1.4.2 IEEE 754 Floating-Point Representation

The **IEEE 754** standard defines:

- **Single precision (32 bits)**: 1 sign bit, 8 exponent bits (bias 127), 23 mantissa bits.
- **Double precision (64 bits)**: 1 sign bit, 11 exponent bits (bias 1023), 52 mantissa bits.

The value is $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent}-\text{bias}}$. Special values include zero, infinity, and NaN (Not a Number).

1.4.3 Character Encoding

- **ASCII**: 7-bit code (0–127) for English characters, often stored in 8 bits with the high bit zero.
- **UTF-8**: Variable-length encoding (1 to 4 bytes) for Unicode, backward compatible with ASCII.
- **UTF-16**: 2 or 4 bytes per character, used in some systems (e.g., Java, Windows).

1.5 Conclusion

Primitive data types and their memory representation form the foundation upon which all data structures are built. The choice of representation affects the range and precision of values, memory consumption, and performance. Understanding binary encoding, endianness, alignment, and standards like Two's Complement and IEEE 754 enables programmers to make informed decisions when designing data structures and to avoid subtle bugs related to data representation.

Tutorial Questions

1. List the four main categories of primitive data types and give a typical size (in bits) for each.
2. Explain the difference between signed and unsigned integer types. What is the range of an 8-bit signed integer in Two's Complement representation?
3. Describe the Three components of an IEEE 754 single-precision floating-point number. What is the bias for the exponent?
4. Compare big-endian and little-endian byte ordering. Show how the 32-bit hexadecimal value `0xDEADBEEF` would be stored in memory starting at address `0x2000` for each endianness.
5. What is memory alignment? Why might a compiler insert padding bytes into a structure?
6. Given the 8-bit Two's Complement representation `11110110`, what decimal value does it represent? Show your work.
7. A structure contains a `char` (1 byte), an `int` (4 bytes), and a `short` (2 bytes) in that order. Assuming 4-byte alignment for `int` and 2-byte alignment for `short`, draw the memory layout including any padding bytes. What is the total size of the structure?
8. Why is the Two's Complement representation preferred over sign-magnitude for signed integers in modern computers?

Data Structures

CSC 301

Lecture Notes

Chapter 2: Pointers and Memory Management

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

2	Pointers and Memory Management	2
2.1	Introduction	2
2.2	Pointers and References	2
2.2.1	Pointers	2
2.2.2	References	2
2.2.3	Dereferencing	3
2.3	Memory Allocation: Stack vs. Heap	3
2.3.1	Stack Allocation	3
2.3.2	Heap Allocation	3
2.4	Run-Time Storage Management	3
2.5	Conclusion	3

Chapter 2

Pointers and Memory Management

2.1 Introduction

This chapter explores the fundamental concepts of pointers and memory management, which are critical for understanding how data structures interact with computer memory. Pointers and references enable dynamic memory allocation and efficient data manipulation, while stack and heap allocation strategies determine how memory is organized during program execution. Run-time storage management ensures that memory is allocated and deallocated appropriately, preventing issues such as memory leaks or dangling pointers.

2.2 Pointers and References

2.2.1 Pointers

A **Pointer** is a variable that stores the memory address of another variable or data structure. Unlike direct references to data, pointers allow indirect access, enabling operations such as dynamic memory allocation and traversal of complex data structures. For example, in C, a pointer to an integer is declared as `int *ptr;`, and its value is set using the address-of operator `&`.

2.2.2 References

A **Reference** is a high-level language construct that provides an indirect way to access memory locations, similar to pointers. However, references are typically managed automatically by the language runtime, ensuring safety and preventing null references. In languages like C++, references are declared as `int ref = var;`, where `ref` is tied to the lifetime of `var`.

2.2.3 Dereferencing

Dereferencing a pointer involves accessing the value stored at the memory address it holds. This is achieved using the dereference operator `*`. For instance, if `ptr` points to an integer `x`, `*ptr` retrieves the value of `x`. Improper dereferencing can lead to runtime errors, such as accessing invalid memory.

2.3 Memory Allocation: Stack vs. Heap

2.3.1 Stack Allocation

Stack Allocation is a memory allocation strategy where data is stored in a last-in, first-out (LIFO) structure. It is typically used for local variables and function call management. Stack allocation is fast and automatic, as memory is released when the function exits. However, it has limited size and cannot be resized dynamically.

2.3.2 Heap Allocation

Heap Allocation involves dynamic memory allocation from a pool of memory called the heap. This allows for flexible memory usage, as data can be allocated and deallocated at runtime. Functions like `malloc()` in C or `new` in C++ are used to request memory on the heap. Unlike stack allocation, heap memory must be explicitly freed to avoid memory leaks.

2.4 Run-Time Storage Management

Run-time storage management encompasses the processes of allocating and deallocating memory during program execution. This includes manual management, where the programmer controls memory (e.g., using `free()` in C), and automatic management, such as garbage collection in languages like Java. Efficient run-time management is essential for optimizing performance and preventing memory-related errors.

2.5 Conclusion

Understanding pointers, memory allocation strategies, and run-time storage management is vital for designing efficient data structures. These concepts form the foundation for implementing complex algorithms and managing resources effectively in software development.

Tutorial Questions

1. Define "Pointers" using the terminology registry.
2. Compare stack and heap allocation in terms of memory management.

3. What is the difference between a pointer and a reference?
4. Explain the concept of memory leaks and how they can occur.
5. Write a C function that allocates memory on the heap and returns a pointer.
6. Discuss the implications of improper memory management.
7. How does garbage collection differ from manual memory management?
8. Provide an example of a data structure that uses linked structures.

Data Structures

CSC 301

Lecture Notes

Chapter 3: Basic Data Structures: Arrays, Records, and Strings

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

3	Basic Data Structures: Arrays, Records, and Strings	2
3.1	Introduction	2
3.1.1	Arrays	2
3.1.2	Records	3
3.1.3	Strings	3
3.1.4	String Processing	3
3.1.5	Conclusion	4

Chapter 3

Basic Data Structures: Arrays, Records, and Strings

[TERMINOLOGY] [/TERMINOLOGY]

[CHAPTER]

3.1 Introduction

This chapter introduces three fundamental data structures that serve as the building blocks for more complex structures: **Arrays**, **Records**, and **Strings**. Each of these structures provides a distinct way to organize and access data in memory, and each is closely tied to the underlying memory representation of the data it holds. Understanding these structures is essential before proceeding to the linked structures, stacks, queues, and trees discussed in later chapters.

3.1.1 Arrays

An **array** is a collection of elements stored in contiguous memory locations, where each element can be accessed directly by its index. The index is typically an integer that ranges from zero to one less than the total number of elements. Because the elements are stored contiguously, the address of any element can be computed in constant time using the base address of the array and the index of the desired element.

For example, consider an array of five integers. The element at index zero occupies the first memory location, the element at index one occupies the next, and so on. This layout allows random access to any element in constant time, denoted as $O(1)$. However, the size of a static array is fixed at the time of creation, and inserting or removing elements in the middle of the array requires shifting subsequent elements, which takes linear time.

Arrays can be allocated on the stack or on the heap. Stack allocation is suitable for arrays whose size is known at compile time and whose lifetime is limited to the scope in which

they are declared. Heap allocation is used when the size is not known until run time or when the array must outlive the function that created it.

3.1.2 Records

A **record** is a composite data structure that groups together a fixed number of fields, each of which may have a different data type. Unlike an array, where all elements share the same type, a record allows heterogeneous data to be stored together under a single name. Each field within a record is identified by a name or a tag, and the fields are accessed individually rather than by a numeric index.

For instance, a record representing a student might contain a field for the student's name, a field for the student's age, and a field for the student's grade point average. The fields are stored in a fixed order, and the memory layout of the record is determined by the order and types of its fields. Memory alignment and padding may be applied to ensure that each field begins at an address that is a multiple of its size, which can affect the total size of the record.

Records are useful for modeling real-world entities that have multiple attributes. They are sometimes referred to as structs in certain programming languages, but the underlying concept remains the same.

3.1.3 Strings

A **string** is a sequence of characters used to represent text. In most programming languages, strings are stored as arrays of characters, with an additional convention to mark the end of the sequence. In some languages, a special null character is appended to the end of the character array to indicate termination. In other languages, the length of the string is stored separately as part of the string object.

Strings can be either mutable or immutable. In a mutable string, the characters can be modified after the string is created. In an immutable string, any operation that appears to modify the string actually creates a new string object. The choice between mutability and immutability affects both the performance and the safety of string operations.

3.1.4 String Processing

String processing refers to the set of operations and algorithms used to manipulate and analyze text data. Common operations include concatenation, which joins two strings into one; substring extraction, which retrieves a contiguous portion of a string; and searching, which locates a pattern within a string. More advanced operations include pattern matching using regular expressions, sorting of collections of strings, and transformation of text according to specified rules.

Many programming languages provide built-in functions or library routines for these operations. For example, a concatenation function might take two strings as input and return a new string that contains the characters of the first followed by the characters of

the second. A search function might take a string and a pattern as input and return the index at which the pattern first appears in the string.

3.1.5 Conclusion

Arrays, records, and strings are foundational data structures that provide efficient ways to store and access collections of data. Arrays offer constant-time indexed access to elements of the same type. Records allow heterogeneous data to be grouped under a single name. Strings provide a specialized representation for text that supports a rich set of processing operations. Together, these structures form the basis for the more advanced data structures that are explored in subsequent chapters.

Tutorial Questions

1. Define an array and explain how the index of an element is used to compute its memory address.
2. Compare stack allocation and heap allocation in the context of array storage.
3. Describe the difference between an array and a record in terms of element types and access methods.
4. Explain how memory alignment and padding can affect the size of a record in memory.
5. What is the difference between a mutable string and an immutable string?
6. List four common operations in string processing and briefly describe each.
7. Given an array of integers containing the values 7, 2, 9, 4, and 1, what value is stored at index 3 if indexing begins at zero?

[/CHAPTER]

Data Structures

CSC 301

Lecture Notes

Chapter 4: Stacks, Queues, and Linked Structures

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

4	Stacks, Queues, and Linked Structures	2
4.1	Introduction	2
4.1.1	Stacks	2
4.1.2	Queues	2
4.1.3	Linked Structures	2
4.1.4	Implementation Strategies for Stacks and Queues	3
4.1.5	Comparison of Stack and Queue Implementations	3
4.1.6	Conclusion	3

Chapter 4

Stacks, Queues, and Linked Structures

4.1 Introduction

Stacks and queues are abstract data types that model collection behaviors with specific ordering constraints. A stack follows the last-in, first-out (LIFO) principle, while a queue follows the first-in, first-out (FIFO) principle. These structures are fundamental for managing task scheduling, expression evaluation, and breadth-first traversal in algorithms.

4.1.1 Stacks

A stack is a linear collection that supports two primary operations: **push**, which adds an element to the top, and **pop**, which removes the top element. The most recent element inserted is the first one removed, adhering to the LIFO order. Stacks may be implemented using an array or a linked list, each with distinct memory characteristics.

4.1.2 Queues

A queue is a linear collection that supports **enqueue**, which adds an element to the rear, and **dequeue**, which removes the front element. The earliest element inserted is the first one removed, satisfying the FIFO order. Like stacks, queues can be realized with contiguous arrays or with linked structures.

4.1.3 Linked Structures

Linked structures consist of **nodes** connected by **pointers** or **references**. Each node typically stores data and a link to another node, allowing the overall structure to grow dynamically without requiring contiguous memory. This contrasts with arrays, where elements are stored in adjacent memory locations.

4.1.4 Implementation Strategies for Stacks and Queues

Two primary implementation strategies exist:

- **Array-based implementation:** Memory is allocated as a contiguous block. Push and pop (or enqueue and dequeue) operations are $O(1)$ when the structure does not exceed capacity, but resizing may be required, leading to occasional $O(n)$ cost.
- **Linked-list-based implementation:** Each element is a node allocated on the heap. Insertions and deletions are $O(1)$ without resizing, but each operation incurs additional overhead for pointer manipulation and memory allocation.

The choice between these strategies affects time complexity, memory usage, and flexibility, especially when the size of the collection is not known in advance.

4.1.5 Comparison of Stack and Queue Implementations

Arrays provide constant-time random access, which benefits stack **push** and **pop** operations, but they limit dynamic resizing. Linked lists eliminate the need for resizing and allow unlimited growth, at the cost of extra memory for pointers and slower traversal. Queues implemented with a circular buffer (array-based) achieve $O(1)$ enqueue and dequeue, while a linked-list queue maintains $O(1)$ operations without the complexity of buffer wrap-around logic.

4.1.6 Conclusion

Stacks and queues are essential abstract data types whose implementations can be tailored to application requirements. Array-based structures offer speed and simplicity for fixed-size scenarios, whereas linked-list-based structures provide dynamic sizing and flexibility at the expense of additional overhead. Understanding these trade-offs is crucial for effective data structure selection in system design.

Tutorial Questions

1. What principle does a stack follow, and how is it formally expressed?
2. Explain the difference between array-based and linked-list-based implementations of a stack.
3. How does the memory access pattern of a queue differ from that of a stack?
4. If a stack initially empty performs the operations **push 10**, **push 20**, **pop**, **push 30**, what is the value on the top of the stack after the final operation?
5. Define the term **node** as used in linked structures.
6. Compare the time complexity of **enqueue** and **dequeue** operations in an array-based queue versus a linked-list-based queue.

7. What role does **memory alignment** play in the layout of a structure containing both an array and a pointer?
8. Explain why a queue implemented with a linked list may avoid the need for resizing operations.

Data Structures

CSC 301

Lecture Notes

Chapter 5: Tree Data Structures

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

5	Tree Data Structures	2
5.1	Introduction to Tree Data Structures	2
5.1.1	Tree Terminology and Properties	2
5.1.2	Tree Traversal Algorithms	2
5.1.3	Implementation Strategies for Trees	3
5.1.4	Applications of Trees	3
5.2	Conclusion	3

Chapter 5

Tree Data Structures

5.1 Introduction to Tree Data Structures

Trees are hierarchical data structures composed of nodes connected by edges, with a single root node and child nodes forming subtrees. Unlike linear structures such as arrays or linked lists, trees allow efficient representation of hierarchical relationships, making them essential for modeling real-world systems like file systems, organizational charts, and decision trees. This chapter explores tree definitions, traversal algorithms, and implementation strategies for binary trees, binary search trees (BSTs), and balanced trees.

5.1.1 Tree Terminology and Properties

A tree consists of nodes connected by edges. Key terms include: - **Root**: The topmost node with no parent. - **Leaf**: A node with no children. - **Internal Node**: A node with at least one child. - **Subtree**: A tree formed by a node and its descendants. - **Depth**: The number of edges from the root to a node. - **Height**: The maximum depth of any node in the tree.

Trees can be classified by constraints on node degrees. A **binary tree** restricts each node to at most two children (left and right). A **full binary tree** has all nodes with 0 or 2 children. A **complete binary tree** fills all levels except possibly the last, which is filled left to right.

5.1.2 Tree Traversal Algorithms

Traversal algorithms visit nodes in specific orders:

- **Preorder (Root, Left, Right)**: Visit the root, then recursively traverse the left and right subtrees.
- **Inorder (Left, Root, Right)**: Traverse the left subtree, visit the root, then the right subtree. For BSTs, inorder traversal yields nodes in ascending order.

- **Postorder (Left, Right, Root):** Traverse left and right subtrees before visiting the root. Useful for deleting nodes or evaluating expressions.

Example: For a tree with root A, left child B, and right child C, preorder traversal visits $A \rightarrow B \rightarrow C$, while inorder visits $B \rightarrow A \rightarrow C$.

5.1.3 Implementation Strategies for Trees

Trees are typically implemented using node-based structures. A **Node** struct contains data and pointers to child nodes:

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

Binary Search Trees (BSTs)

A BST enforces the property that for any node, all left descendants have values less than the node, and all right descendants have values greater. Insertion and deletion maintain this property: - **Insertion:** Start at the root. If the new value is less than the current node, move left; otherwise, move right. Insert as a leaf. - **Deletion:** Three cases arise: 1. Node is a leaf: Remove it. 2. Node has one child: Replace it with its child. 3. Node has two children: Replace it with its inorder successor (smallest node in the right subtree) and delete the successor.

Balanced Trees

Unbalanced BSTs can degrade to linked lists in the worst case. Balanced trees, such as AVL trees and red-black trees, maintain height $O(\log n)$ through rotations during insertions and deletions. For example, AVL trees use balance factors (differences in subtree heights) to trigger rotations.

5.1.4 Applications of Trees

Trees are widely used in: - **File Systems:** Hierarchical organization of files and directories. - **Expression Parsing:** Abstract syntax trees represent mathematical expressions. - **Database Indexing:** B-trees enable efficient disk-based searches. - **Network Routing:** Spanning trees optimize path selection.

5.2 Conclusion

Trees provide a flexible framework for organizing hierarchical data. Understanding traversal algorithms and implementation strategies is critical for designing efficient data structures. Balanced trees, in particular, ensure optimal performance for dynamic operations.

Tutorial Questions

1. Define a binary tree and explain the difference between a full binary tree and a complete binary tree.
2. Describe the steps for performing an inorder traversal of a binary tree.
3. How does a BST ensure efficient search operations, and what is its worst-case time complexity?
4. Explain the process of deleting a node with two children in a BST.
5. What is the purpose of rotations in AVL trees, and how do they maintain balance?
6. Compare the memory allocation strategies for static arrays and dynamic trees.
7. Write a recursive function to calculate the height of a binary tree.
8. Discuss one application of trees in computer science and explain its relevance.